



AI Challenge - Hướng dẫn

Đạt, Tuấn

Mục lục

1	Về cuộc thi	2
1.1	Thứ tự xử lý mỗi Step	2
1.2	Hành động của Agent	2
1.3	Cơ chế Bom	2
1.4	Vật phẩm (Items)	3
1.5	Điều kiện loại bỏ và Kết thúc	3
2	Rule-based	3
2.1	Cơ chế hoạt động	4
2.2	Ưu điểm	4
2.3	Nhược điểm	4
3	Các thuật toán cơ bản	4
3.1	Đối kháng và Tìm kiếm có đối thủ (Adversarial Search)	4
3.1.1	Thuật toán Minimax	4
3.1.2	Cắt tỉa Alpha-Beta (Alpha-Beta Pruning)	5
4	Học tăng cường	6
4.1	Từ Markov Process đến Model-free Evaluation	7
4.2	Deep Q-Learning	8
4.2.1	Nguồn gốc, lý thuyết nền tảng và bài toán hội tụ	8
4.2.2	Thuật toán chi tiết và mã giả DQN	9
4.3	Policy Gradient Methods	12
4.3.1	Hạn chế của DQN và Triết lý tối ưu trực tiếp chính sách	12
4.3.2	Thuật toán PPO (Proximal Policy Optimization)	13
4.3.3	Đánh giá và so sánh thực tiễn	15
4.4	Self-Play	15
4.4.1	Nguồn gốc và ý tưởng cơ bản	15
4.4.2	Các bước chi tiết và mã giả	16
4.5	Behaviour Cloning	18
4.5.1	Nguồn gốc và ý tưởng cơ bản	18
4.5.2	Các bước chi tiết và mã giả	18
4.5.3	Monte Carlo Tree Search (MCTS)	21
5	Nguồn tham khảo	24



1. Về cuộc thi

Dưới đây là tóm tắt chi tiết về cơ chế game:

1.1 Thứ tự xử lý mỗi Step

Mỗi bước (*step*) của trò chơi được engine xử lý tuần tự theo các giai đoạn sau:

[Thu thập hành động] → [Xử lý di chuyển] → [Đặt bom] → [Giảm timer bom] → [Giải quyết nổ] → [Loại agent] → [Spawn vật phẩm] → [Kiểm tra kết thúc]

1.2 Hành động của Agent

Tại mỗi step, mỗi Agent gửi về một số nguyên tương ứng với hành động:

Giá trị	Hành động	Mô tả
0	STOP	Đứng yên tại vị trí hiện tại
1	LEFT	Di chuyển sang trái 1 ô
2	RIGHT	Di chuyển sang phải 1 ô
3	UP	Di chuyển lên trên 1 ô
4	DOWN	Di chuyển xuống dưới 1 ô
5	PLACE_BOMB	Đặt bom tại vị trí hiện tại

Quy tắc di chuyển:

- Không thể đi vào ô có **Tường** (mã 1) hoặc **Hộp** (mã 2).
- Không thể đi vào ô đã có **Bom** từ các step trước. *Ngoại lệ:* Nếu Agent vừa đặt bom tại ô đó trong step hiện tại thì vẫn có thể di chuyển ra ngoài.
- Nhiều Agent có thể đứng cùng một ô.
- Nếu ≥ 2 Agent cùng bước vào một ô có vật phẩm: Vật phẩm bị hủy, không Agent nào nhận được.

1.3 Cơ chế Bom

Thông số mặc định:

- **Timer:** 7 steps (đếm ngược về 0, nổ khi ≤ 0).
- **Bán kính (Radius):** Bắt đầu là 1, tối đa là 5.
- **Số lượng bom (Capacity):** Bắt đầu là 1, tối đa là 5.

Quy tắc đặt bom:

- Điều kiện: `bombs_left > 0` và ô hiện tại chưa có bom từ step trước.
- **Xử lý trùng lặp (nhiều Agent đặt bom cùng 1 ô):** Ưu tiên bom có bán kính lớn hơn. Nếu bán kính bằng nhau, ưu tiên Agent có ID nhỏ hơn.



- Chỉ người đặt bom cuối cùng (theo quy tắc trên) mới bị trừ `bombs_left`.

Cơ chế nổ:

- Vụ nổ lan theo 4 hướng (Thập tự).
- **Vật cản:** Tường chặn hoàn toàn vụ nổ. Hộp chặn vụ nổ và bị phá hủy. Agent không chặn được vụ nổ.
- **Phản ứng dây chuyền (Chain reaction):** Nếu vụ nổ chạm vào một quả bom khác, quả bom đó sẽ nổ ngay lập tức trong cùng step đó.

1.4 Vật phẩm (Items)

Vật phẩm giúp tăng sức mạnh cho Agent (Radius hoặc Capacity).

- **Từ việc phá hộp:**
 - 30% rơi *Item Radius*
 - 30% rơi *Item Capacity*
 - 40% không rơi gì.
- **Tự động xuất hiện (Auto-spawn):** Tại mỗi ô trống, xác suất P xuất hiện vật phẩm mỗi step là:

$$P = 0.0003 \times \left(\frac{\text{step}}{165} \right)$$

Trong đó: 50% là *Radius*, 50% là *Capacity*.

1.5 Điều kiện loại bỏ và Kết thúc

- **Loại Agent:** Agent bị loại ngay lập tức nếu đứng trong ô bị ảnh hưởng bởi vụ nổ (kể cả tự đặt bom). Agent đã bị loại vẫn để lại các quả bom cũ trên sân.
- **Kết thúc trận đấu:**
 1. Khi chỉ còn ≤ 1 Agent sống sót.
 2. Khi đạt giới hạn 500 steps.

2. Rule-based

Cách tiếp cận này sẽ xây dựng agent thông qua các heuristic được đặt sẵn. Thay vì học từ trải nghiệm, agent chạy theo một tập hợp các quy tắc được lập trình sẵn để xử lý các tình huống khác nhau trên bản đồ.



2.1 Cơ chế hoạt động

Agent đánh giá trạng thái hiện tại (vị trí các vật cản, bom, đối thủ, vật phẩm) và áp dụng một chuỗi điều kiện if-else để quyết định hành động tiếp theo. Ví dụ, nếu phía trước có tường thì không đi, nếu gần đối thủ mà có bom sẵn sàng thì đặt bom, nếu thấy vật phẩm giữa đường thì đi lấy.

Chiến lược thường bao gồm:

- Tìm kiếm tuyến tính để định vị đối thủ gần nhất, vật phẩm có giá trị cao nhất
- Ưu tiên các hành động dựa trên khoảng cách hoặc mức độ nguy hiểm
- Tránh các vùng có bom hoặc tường
- Kiến thức cứng về cách di chuyển an toàn sau khi đặt bom

2.2 Ưu điểm

Dễ triển khai nhanh chóng. Không cần dữ liệu huấn luyện hay thời gian tính toán để thiết lập mô hình. Agent có thể chạy được ngay, và hành vi của agent có thể được giải thích dễ dàng.

2.3 Nhược điểm

Quy tắc được code cứng không thể tổng quát hóa cho các tình huống khác nhau. Agent sẽ thất bại trước những tình huống ngoài phạm vi các if-else đã viết. Agent không học từ thất bại mà chỉ hành động theo một heuristic cứng nhắc.

3. Các thuật toán cơ bản

3.1 Đối kháng và Tìm kiếm có đối thủ (Adversarial Search)

Bomberland là một trò chơi chiến thuật theo lượt với sự tham gia của 4 agent trên một bản đồ lưới 13×13 . Do sự hiện diện của các đối thủ cạnh tranh trực tiếp, bài toán không chỉ dừng lại ở việc tìm đường đi ngắn nhất mà còn là việc dự đoán và đối phó với hành động của các agent khác. Các thuật toán tìm kiếm có đối thủ như Minimax là lựa chọn cốt lõi để giải quyết vấn đề này.

3.1.1 Thuật toán Minimax

Minimax là một thuật toán đệ quy được sử dụng để chọn hành động tối ưu cho một người chơi (Max) với giả định rằng đối thủ (Min) cũng đang chơi tối ưu để chống lại họ. Mặc dù Bomberland có 4 agent (có thể yêu cầu mở rộng thành thuật toán Max-N), một cách tiếp cận đơn giản và hiệu quả là coi bài toán dưới dạng "1 vs All" (Agent của ta là Max, môi trường và 3 agent còn lại gộp chung thành Min).

Mỗi bước trong game, agent phải chọn một hành động $a \in \{0, 1, 2, 3, 4, 5\}$ tương ứng với Đứng yên, Trái, Phải, Lên, Xuống, và Đặt bom.

Công thức tính giá trị của một trạng thái s trong cây Minimax cơ bản (2 người chơi):

$$V(s) = \max_{a \in A} \min_{a' \in A'} V(\text{Result}(s, a, a'))$$

Trong đó:

- A là tập hành động của Agent (Max).
- A' là tập hành động dự kiến của đối thủ (Min).
- $\text{Result}(s, a, a')$ là trạng thái tiếp theo sau khi các hành động được thực hiện.

Hàm đánh giá Heuristic $E(s)$ trong Bomberland:

Do giới hạn thời gian xử lý (inference timeout) của mỗi bước chỉ là 100ms, thuật toán không thể duyệt cây Minimax đến cuối ván đấu (tối đa 500 bước). Vì vậy, việc duyệt cây phải dừng lại ở một độ sâu d nhất định, và trạng thái tại các nút lá được đánh giá thông qua một hàm Heuristic $E(s)$.

Một hàm Heuristic tốt cho Bomberland cần cân nhắc các yếu tố:

- **An toàn:** Phạt điểm nặng nếu agent nằm trong tầm nổ của bom ở các step tiếp theo (bom có timer đếm ngược từ 7).
- **Phá hộp (Farming):** Thưởng điểm khi đặt bom ở vị trí có thể phá hủy nhiều hộp (Box - ký hiệu 2), vì hộp có 60% tỷ lệ rơi vật phẩm (30% Radius và 30% Capacity).
- **Khoảng cách:** Đánh giá khoảng cách Manhattan hoặc A^* từ agent đến vật phẩm an toàn gần nhất (Radius hoặc Capacity) hoặc vùng an toàn để né bom.

3.1.2 Cắt tỉa Alpha-Beta (Alpha-Beta Pruning)

Việc duyệt toàn bộ cây Minimax sẽ bùng nổ tổ hợp, đặc biệt khi mỗi agent có tới 6 hành động có thể thực hiện tại mỗi step. Cắt tỉa Alpha-Beta là một kỹ thuật tối ưu hóa cho Minimax giúp giảm thiểu đáng kể số lượng nút cần đánh giá mà không làm thay đổi kết quả cuối cùng.

Thuật toán duy trì hai giá trị trong quá trình duyệt cây:

- α : Giá trị tốt nhất (cao nhất) mà Max có thể đảm bảo đạt được trên con đường hiện tại.
- β : Giá trị tốt nhất (thấp nhất) mà Min có thể đảm bảo đạt được trên con đường hiện tại.

Điều kiện cắt tỉa:

Tại bất kỳ nút Min nào, nếu giá trị hiện tại $v \leq \alpha$, Max sẽ không bao giờ chọn đi theo nhánh này (vì Max đã có một lựa chọn α tốt hơn ở nơi khác). Nhánh này có thể bị cắt bỏ (Pruning). Tương tự, tại nút Max, nếu $v \geq \beta$, Min sẽ không bao giờ cho phép Max đạt được trạng thái này.

Công thức cập nhật:

- Tại nút Max: $\alpha = \max(\alpha, v)$

- Tại nút Min: $\beta = \min(\beta, v)$
- Cắt tỉa xảy ra khi: $\alpha \geq \beta$

Áp dụng thực tế trong cuộc thi:

Với thời gian giới hạn khắt khe 100ms/step, Alpha-Beta Pruning kết hợp với kỹ thuật *Iterative Deepening* (tăng dần độ sâu tìm kiếm cho đến khi hết thời gian) là cực kỳ cần thiết. Hơn nữa, ta có thể kết hợp việc sắp xếp các hành động (Move Ordering) để tăng hiệu quả cắt tỉa. Ví dụ: ưu tiên xét các hành động "Di chuyển đến vùng an toàn" trước hành động "Đặt bom" nếu trạng thái hiện tại đang bị đe dọa bởi bom có timer sắp về 0. Điều này giúp nhánh Alpha-Beta nhanh chóng tìm thấy các giá trị α, β tốt, từ đó cắt tỉa được nhiều nhánh vô ích hơn.

4. Học tăng cường

Trong bối cảnh phát triển các tác nhân thông minh cho trò chơi, học tăng cường (Reinforcement Learning - RL) nổi lên như một phương pháp tiếp cận mạnh mẽ, cho phép agent tự tìm kiếm chính sách hành động tối ưu thông qua quá trình "thử và sai" (trial and error) nhờ tương tác trực tiếp với môi trường. Khác với các hướng tiếp cận truyền thống, tác nhân RL không cần một kịch bản lập trình sẵn hay quá trình tiến hóa nhân tạo; thay vào đó, nó tự điều chỉnh hành vi dựa trên tín hiệu phần thưởng (reward) nhận được sau mỗi hành động với mục tiêu tối cao là tối đa hóa tổng phần thưởng kỳ vọng trong tương lai.

Về mặt toán học, bài toán này được chuẩn hóa thông qua khung lý thuyết Markov Decision Process (MDP), mở rộng từ chuỗi Markov và Markov Reward Process (MRP) bằng việc tích hợp thêm yếu tố hành động chủ quan của tác nhân. Mô hình MDP được định nghĩa bởi một bộ các đại lượng cơ bản:

- S : Tập hợp hữu hạn các trạng thái khả dĩ của môi trường.
- A : Tập hợp hữu hạn các hành động mà agent có thể thực hiện.
- $V(s)$ (State Value Function): Hàm giá trị trạng thái, biểu diễn phần thưởng kỳ vọng tích lũy khi tác nhân bắt đầu từ trạng thái s .
- $Q(s, a)$ (State-Action Value Function): Hàm giá trị trạng thái - hành động, lượng hóa lợi ích kỳ vọng dài hạn nếu tác nhân thực hiện hành động a ngay tại trạng thái s .
- $P(s'|s, a)$ (State Transition Probability): Xác suất môi trường chuyển sang trạng thái kế tiếp s' sau khi agent thực hiện hành động a tại trạng thái s .
- $R(s, a)$ (Reward Function): Phần thưởng kỳ vọng nhận được khi thực hiện hành động a tại trạng thái s , tức $R(s, a) = \mathbb{E}[r_t | s_t = s, a_t = a]$.
- G_t : Tổng phần thưởng có chiết khấu từ thời điểm t , thường viết $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$, trong đó γ điều chỉnh mức ưu tiên giữa lợi ích hiện tại và tương lai.
- $\pi(a|s)$ (Policy): Chính sách hành động, xác định xác suất tác nhân lựa chọn hành động a khi đối mặt với trạng thái s . Chính sách này có thể mang tính ngẫu nhiên

(Stochastic policy) ký hiệu là $\pi(a|s)$ hoặc mang tính tất định (Deterministic policy) ký hiệu là $\pi(s) = a$.

4.1 Từ Markov Process đến Model-free Evaluation

Trước khi đi vào Deep Q-Learning, cần nắm chuỗi xây dựng lý thuyết từ Markov Process (MP), Markov Reward Process (MRP), đến Markov Decision Process (MDP). Markov Process chỉ mô tả chuỗi trạng thái chuyển đổi theo xác suất cố định, chưa có hành động và chưa có phần thưởng. Giả định Markov yêu cầu trạng thái hiện tại đã chứa đủ thông tin cần thiết từ quá khứ, nên tương lai chỉ phụ thuộc vào trạng thái hiện tại thay vì toàn bộ lịch sử.

Khi bổ sung phần thưởng, ta có Markov Reward Process. Giá trị trạng thái được định nghĩa là kỳ vọng của tổng phần thưởng chiết khấu:

$$V(s) = \mathbb{E}[G_t | s_t = s] = \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t = s].$$

Nếu biết xác suất chuyển trạng thái $P(s'|s)$ và phần thưởng $R(s)$, phương trình Bellman cho MRP là:

$$V(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s) V(s').$$

Dạng ma trận tương ứng là $V = R + \gamma PV$, suy ra $V = (I - \gamma P)^{-1}R$. Tuy nhiên, cách giải trực tiếp này có chi phí lớn và đòi hỏi biết trước mô hình môi trường, nên trong thực tế thường chuyển sang quy hoạch động hoặc các phương pháp học từ mẫu.

MDP mở rộng MRP bằng cách thêm hành động của agent. Khi biết trước $R(s, a)$ và $P(s'|s, a)$, agent có thể tìm chính sách tốt bằng cách chọn hành động làm cực đại giá trị kỳ vọng trong phương trình Bellman tối ưu. Với hàm giá trị hành động, nguyên tắc ra quyết định trở nên trực tiếp hơn: tại trạng thái s , chọn hành động a sao cho $Q(s, a)$ lớn nhất.

Trong các trò chơi như Bomberman, ta thường không biết chính xác R và P của toàn bộ môi trường. Vì vậy cần dùng **model-free policy evaluation**, tức học giá trị từ dữ liệu tương tác thay vì từ mô hình chuyển trạng thái có sẵn. Hai hướng cơ bản là:

- **Monte Carlo (MC):** Ước lượng $V(s)$ hoặc $Q(s, a)$ bằng trung bình phần thưởng thu được sau nhiều episode hoàn chỉnh. MC không cần biết $P(s'|s, a)$, nhưng phải chờ episode kết thúc nên cập nhật chậm và tốn thời gian nếu ván chơi dài.
- **Temporal Difference (TD):** Cập nhật ngay sau từng bước bằng cách kết hợp phần thưởng quan sát được với ước lượng hiện tại của trạng thái kế tiếp. TD dùng bootstrapping, có phương sai thấp hơn MC và phù hợp với tiến trình có thể kéo dài rất lâu, nhưng có thể tạo bias do dựa trên ước lượng chưa hoàn hảo.

Tóm lại, Dynamic Programming cần biết mô hình chuyển trạng thái, Monte Carlo học từ episode hoàn chỉnh, còn Temporal Difference kết hợp ưu điểm của cả hai: học trực tiếp từ trải nghiệm như MC nhưng cập nhật từng bước như DP.

4.2 Deep Q-Learning

4.2.1 Nguồn gốc, lý thuyết nền tảng và bài toán hội tụ

Thuật toán Q-Learning ban đầu được nghiên cứu và phát triển bởi Christopher Watkins vào những năm 1980. Bản chất của phương pháp này là học một hàm giá trị hành động tối ưu nhằm chỉ ra mức độ "tốt" của một quyết định cụ thể trong một hoàn cảnh cụ thể. Trọng tâm toán học của thuật toán xoay quanh phương trình Bellman dưới dạng cập nhật gia tăng sau mỗi bước chuyển đổi:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Trong đó, r là phần thưởng tức thời nhận được từ môi trường; s' là trạng thái kế tiếp; $\alpha \in (0, 1]$ đóng vai trò là tốc độ học (learning rate); và hằng số chiết khấu $\gamma \in [0, 1]$ xác định mức độ ưu tiên của tác nhân đối với các phần thưởng hiện tại so với lợi ích lâu dài trong tương lai. Thành phần nằm trong dấu ngoặc vuông $[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ chính là sai số hiệu số thời gian (TD error), phản ánh khoảng cách giữa ước lượng hiện tại và mục tiêu TD cải tiến (TD target).

Điều kiện hội tụ trong không gian tabular: Khi được biểu diễn dưới dạng bảng rời rạc (Tabular Settings) - tức là mỗi cặp (s, a) là một ô nhớ độc lập trong bộ dữ liệu - thuật toán Q-Learning được chứng minh một cách nghiêm ngặt rằng sẽ hội tụ chắc chắn về hàm giá trị tối ưu $Q^*(s, a)$, vượt qua các phương pháp ước lượng khác như Monte Carlo vốn đòi hỏi phải chạy hết toàn bộ episode mới có thể cập nhật, miễn là thỏa mãn đồng thời hai điều kiện nền tảng:

1. **Chính sách thỏa mãn thuộc tính GLIE (Greedy in the Limit of Infinite Exploration):** Đảm bảo rằng tất cả các cặp trạng thái - hành động đều được tác nhân ghé thăm vô hạn lần ($\lim_{i \rightarrow \infty} N_i(s, a) \rightarrow \infty$) để không bỏ sót nghiệm, đồng thời chính sách hành vi của tác nhân phải tiệm cận về chính sách tham lam hoàn toàn ($\lim_{i \rightarrow \infty} \pi(a|s) \rightarrow \arg \max_a Q(s, a)$ với xác suất bằng 1). Một chiến lược thực tế phổ biến để đạt được GLIE là áp dụng ϵ -greedy và giảm dần giá trị ϵ theo thời gian theo tỷ lệ $\epsilon_i = 1/i$.
2. **Tốc độ học α_t tuân thủ các điều kiện Robbins-Monro:** Chuỗi các hệ số học phải đủ lớn để triệt tiêu ảnh hưởng của các giá trị khởi tạo ban đầu nhưng cũng phải đủ nhỏ để triệt tiêu các nhiễu loạn ngẫu nhiên từ môi trường:

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \text{và} \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Một ví dụ điển hình đáp ứng quy luật này là đặt $\alpha_t = 1/T$.

Deadly Triad và sự chuyển dịch sang Deep DQN: Dù có bảo chứng hội tụ trong không gian tabular, việc ứng dụng bảng Q vào các môi trường thực tế như Bomberman hay Atari là bất khả thi, bởi không gian trạng thái quá lớn khiến bảng lưu trữ phình to vô hạn. Giải pháp tất yếu là chuyển sang xấp xỉ giá trị bảng bằng một mạng nơ-ron có tham số trọng số θ , gọi là $Q(s, a; \theta)$.

Tuy nhiên, việc tích hợp mạng nơ-ron vào Q-Learning sẽ kích hoạt hiện tượng mất ổn định và phân kỳ nghiêm trọng được các nhà nghiên cứu gọi là Deadly Triad. Sự mất ổn định này xảy ra khi hệ thống học hội tụ đồng thời 3 yếu tố:

1. **Function Approximation (Xấp xỉ hàm):** Sử dụng mô hình tham số (như mạng nơ-ron đại lượng lớn) thay cho bảng lưu rời rạc.
2. **Bootstrapping (Tự lấy điểm tựa):** Cập nhật ước lượng hiện tại dựa trên một ước lượng khác của trạng thái kế tiếp (TD learning) thay vì sử dụng phần thưởng thực tế trọn vẹn.
3. **Off-policy learning (Học ngoài chính sách):** Huấn luyện và đánh giá một chính sách mục tiêu tối ưu tối đa trong khi tác nhân lại đang hành động theo một chính sách hành vi khám phá khác.

Để chế ngự "Bộ ba chết chóc", kiến trúc mạng Deep Q-Network (DQN) hiện đại bắt buộc phải tích hợp hai cơ chế then chốt nhằm ổn định hóa quá trình hội tụ đồ thị học:

- **Experience Replay (Bộ nhớ phát lại trải nghiệm):** Trong quá trình chơi game liên tục, dữ liệu thu thập được mang tính tương quan chuỗi rất mạnh do các khung hình kế tiếp hầu như giống hệt nhau, vi phạm giả định dữ liệu độc lập và phân phối chuẩn (i.i.d) của học có giám sát. DQN giải quyết bằng cách lưu trữ các bộ chuyển đổi (s, a, r, s') vào một bộ đệm lưu trữ cố định (Replay Buffer) D . Khi cập nhật mạng, ta lấy ra ngẫu nhiên từng batch nhỏ (minibatch) từ bộ đệm này để phá vỡ tính tương quan tuần tự.
- **Fixed Q-Targets (Mạng mục tiêu cố định):** Nếu chỉ dùng một mạng nơ-ron duy nhất, mục tiêu học (TD target) sẽ liên tục thay đổi sau mỗi bước cập nhật trọng số, giống như việc ta vừa chạy bắn cung vừa dịch chuyển tấm bia. DQN khắc phục bằng cách thiết lập một mạng nơ-ron độc lập thứ hai gọi là Target Network với bộ tham số θ^- biệt lập. Trọng số θ^- sẽ được giữ đóng băng cố định và chỉ được đồng bộ, sao chép lại từ mạng chính θ một cách định kỳ sau mỗi chu kỳ C bước di chuyển. Để lại sự ổn định này, nhược điểm về mặt tài nguyên là hệ thống yêu cầu dung lượng bộ nhớ tăng gấp đôi để lưu giữ song song hai mô hình tham số.

Bên cạnh đó, biến thể tinh chỉnh **Double DQN (DDQN)** thường được triển khai nhằm giải quyết triệt để tật xấu cố hữu của Q-Learning là xu hướng đánh giá quá cao giá trị hành động (overestimation bias). DDQN thực hiện điều này bằng cách tách biệt vai trò: dùng trọng số mạng chính θ để lựa chọn hành động tham lam tốt nhất ở trạng thái tiếp theo, nhưng lại dùng trọng số của mạng mục tiêu θ^- để lượng hóa chính xác giá trị của hành động đó.

4.2.2 Thuật toán chi tiết và mã giả DQN

1. **Khởi tạo cấu trúc:** Tạo lập mạng nơ-ron Q-network chính với bộ tham số ngẫu nhiên θ và sao chép cấu trúc sang mạng mục tiêu target network với trọng số $\theta^- \leftarrow \theta$. Cài đặt bộ đệm lưu trữ trải nghiệm D với kích thước tối đa cho phép.
2. **Chu trình huấn luyện qua các Episode:** Với mỗi vòng lặp trò chơi:
 - Tiếp nhận trạng thái ban đầu s trực tiếp từ môi trường.

- Duy trì vòng lặp cho đến khi trạng thái đạt điều kiện kết thúc (terminal):

- (a) Ra quyết định chọn hành động a tuân theo cơ chế ϵ -greedy policy:

$$a = \begin{cases} \arg \max_{a'} Q(s, a'; \theta) & \text{với xác suất } 1 - \epsilon \\ \text{hành động chọn ngẫu nhiên hoàn toàn} & \text{với xác suất } \epsilon \end{cases}$$

đảm bảo phân phối hợp lý giữa khai thác tri thức (exploit) và khám phá không gian mới (explore).

- (b) Thực thi hành động a trong môi trường, thu hồi tín hiệu phần thưởng tức thời r cùng trạng thái tiếp diễn s' .
- (c) Đóng gói và lưu trữ bộ tứ trải nghiệm (s, a, r, s') vào bộ đệm Replay Buffer D .
- (d) Trích xuất ngẫu nhiên một minibatch chứa các bộ mẫu đồng dạng từ D để tính toán loss độc lập.
- (e) Xác định giá trị mục tiêu huấn luyện y_i cho từng mẫu trong minibatch:

$$y_i = \begin{cases} r_i & \text{nếu } s' \text{ là trạng thái kết thúc (terminal)} \\ r_i + \gamma \max_{a'} Q(s'_i, a'; \theta^-) & \text{nếu } s' \text{ là trạng thái tiếp diễn} \end{cases}$$

- (f) Cập nhật bộ tham số mạng chính θ thông qua thuật toán Lan truyền ngược và Gradient Descent nhằm cực tiểu hóa hàm mất mát bình phương sai số trung bình (MSE Loss):

$$\text{Loss} = \frac{1}{|B|} \sum_{i \in B} [y_i - Q(s_i, a_i; \theta)]^2$$

- (g) Chuyển đổi trạng thái hiện tại sang trạng thái tiếp theo: $s \leftarrow s'$.

- **Đồng bộ định kỳ:** Sau khi trải qua một số lượng C bước cập nhật (hoặc số lượng episode chỉ định), tiến hành cập nhật lại mạng mục tiêu bằng cách sao chép nguyên bản tham số: $\theta^- \leftarrow \theta$.

Mã giả DQN chi tiết dưới dạng Python:

```
1 Initialize Q-network theta and target network theta_minus
2 Initialize replay buffer D = []
3 for each episode:
4     s = initial_state()
5     while not terminal(s):
6         if random() < epsilon:
7             a = random_action()
8         else:
9             a = argmax_a Q(s, a; theta)
10
11     r, s_prime = environment.step(a)
```

```
12     D.append((s, a, r, s_prime))
13
14     if len(D) > batch_size:
15         batch = sample_minibatch(D, batch_size)
16         loss = 0
17         for (s_i, a_i, r_i, s_prime_i) in batch:
18             if terminal(s_prime_i):
19                 target = r_i
20             else:
21                 # Su dung mang target network theta_minus
22                 # de tinh muc tieu on dinh
23                 target = r_i + gamma * max_a Q(s_prime_i,
24                     a; theta_minus)
25                 loss += (target - Q(s_i, a_i; theta))**2
26
27         Update theta using gradient descent on loss
28
29     s = s_prime
30
31     if episode % update_interval == 0:
32         theta_minus = theta
```

Code 1. Deep Q-Learning pseudocode

Khi đặt thuật toán vào môi trường cụ thể như trò chơi Bomberman, việc cấu hình trạng thái s có thể sử dụng mảng ma trận pixel thô trích xuất từ màn hình trò chơi hoặc sử dụng một vector đặc trưng được thiết kế thủ công tinh gọn (bao gồm tọa độ của tác nhân, vị trí đặt bom, khoảng cách đến kẻ địch hay các ô vật phẩm nâng cấp). Hệ thống phân phối phần thưởng có thể được quy định mẫu như sau: +10 điểm nếu giành chiến thắng chung cuộc, -10 điểm nếu tác nhân bị tử trận, +1 điểm khích lệ duy trì sự sống sót qua mỗi bước di chuyển, và +5 điểm thưởng khi ăn được vật phẩm.

Đánh giá tổng quan: Lợi thế vượt trội của cấu trúc DQN là năng lực tự động khai phá các cấu trúc không gian hình ảnh đặc trưng phức tạp từ nguồn dữ liệu thô mà không cần sự can thiệp thủ công từ chuyên gia. Tuy nhiên, hạn chế lớn nhất của nó là cực kỳ tiêu tốn tài nguyên dữ liệu huấn luyện, tốc độ tối ưu rất chậm, và quá trình huấn luyện nhạy cảm cao, dễ rơi vào trạng thái bất ổn định nếu các siêu tham số (như kiến trúc mạng, learning rate, kích thước replay buffer) không được thiết lập chuẩn xác.

Hơn thế nữa, lập trình viên cần đặc biệt lưu ý bẫy **"Reward Hacking"** trong giai đoạn thiết kế hàm thưởng. Đây là hiện tượng tác nhân phát hiện ra kẽ hở toán học để lặp đi lặp lại một hành vi vô nghĩa nào đó nhằm trục lợi điểm số ở mức an toàn thay vì thực hiện đúng ý đồ chiến lược ban đầu của nhà thiết kế. Ví dụ cụ thể trong Bomberman: do tác nhân ở giai đoạn đầu chưa học được kỹ năng né tránh sóng nổ, việc tự ý di chuyển đặt bom thường dẫn đến tự sát và bị phạt điểm nặng (-10). Hệ quả là mạng nơ-ron sẽ chọn giải pháp tiêu cực: điều khiển tác nhân đứng bất động tuyệt đối ở góc an toàn cho đến khi hết giờ trận đấu nhằm bảo toàn điểm số ở mức chấp nhận được (+1 điểm sống sót mỗi bước), từ chối hoàn toàn việc chơi game chủ động.

4.3 Policy Gradient Methods

4.3.1 Hạn chế của DQN và Triết lý tối ưu trực tiếp chính sách

Mặc dù DQN giải quyết rất tốt bài toán không gian trạng thái lớn, thuật toán này lại bỏ lỡ những điểm yếu chí mạng khi đối mặt với hai kịch bản môi trường đặc thù:

1. **Không gian hành động liên tục (Continuous Action Spaces):** Ví dụ như việc điều khiển góc quay vô lăng chính xác của xe tự hành hoặc lực đẩy liên tục của khớp robot. DQN đòi hỏi phải tính toán toán tử $\max_a Q(s, a)$, một tác vụ cực kỳ tốn kém và bất khả thi khi tập hành động là vô hạn và liên tục.
2. **Chính sách tối ưu bắt buộc phải mang tính ngẫu nhiên (Stochastic Policies):** Trong các trò chơi đối kháng có thông tin bất toàn hoặc mang tính triệt tiêu chiến thuật (như oẳn tù tì), bất kỳ một chính sách tất định nào cũng dễ dàng bị đối phương bắt bài. Chính sách tối ưu lúc này bắt buộc phải là một phân phối xác suất ngẫu nhiên cân bằng, điều mà cơ chế tham lam argmax của DQN không thể cung cấp ổn định.

Để vượt qua giới hạn đó, họ phương pháp **Policy Gradient Methods** được nghiên cứu và phát triển mạnh mẽ từ những năm 1990. Thay vì đi đường vòng bằng cách học hàm giá trị Q rồi suy diễn ra hành động, triết lý cốt lõi của Policy Gradient là sử dụng mạng nơ-ron trực tiếp mô hình hóa và tối ưu hóa chính sách $\pi(a|s; \theta)$ (với θ là các tham số trọng số quy định xác suất sinh hành động). Để cập nhật bằng gradient descent, chính sách $\pi(a|s; \theta)$ cần khả vi theo θ , đồng thời gradient $\nabla_{\theta} \pi(a|s; \theta)$ phải tồn tại và hữu hạn.

Mục tiêu cốt lõi của chúng ta là tìm bộ tham số θ để tối đa hóa hàm mục tiêu hiệu năng kỳ vọng $J(\theta)$. Trong episodic case, có trạng thái dừng và bỏ qua discounting để đơn giản, $J(\theta)$ có thể hiểu là kỳ vọng tổng reward của toàn bộ episode do chính sách hiện tại sinh ra. Nhờ vào **Định lý Policy Gradient**, việc tính toán gradient của hàm hiệu năng theo tham số θ được quy giản về dạng biểu thức toán học rất đẹp mà không phụ thuộc vào đạo hàm của hàm chuyển trạng thái môi trường vô hình:

$$\nabla_{\theta} J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi(a|s; \theta) \cdot A(s, a)]$$

Trong biểu thức trên, đại lượng $A(s, a)$ chính là hàm Lợi thế (Advantage function). Hàm này đóng vai trò như một chiếc cân đo lường, đánh giá xem hành động a vừa được thực hiện tốt hơn hay tệ hơn mức kỳ vọng trung bình của trạng thái s hiện tại:

$$A(s, a) = Q(s, a) - V(s)$$

Nếu một hành động mang lại kết quả tốt hơn trung bình ($A(s, a) > 0$), gradient sẽ kéo tham số θ dịch chuyển theo hướng làm tăng xác suất chọn lại hành động đó trong tương lai và ngược lại.

Có thể phân biệt ba mức triển khai Policy Gradient trước khi đi đến PPO:

- **All-actions method:** Nếu có thể xét toàn bộ không gian hành động, ta cập nhật chính sách bằng cách cộng đóng góp gradient của mọi hành động theo xác suất của chúng. Cách này rõ về mặt lý thuyết nhưng tốn kém khi số hành động lớn.

- **REINFORCE:** Lấy mẫu hành động từ chính sách hiện tại rồi cập nhật theo quỹ đạo đã quan sát. Dạng trực giác của cập nhật là tăng xác suất các hành động dẫn đến return cao và giảm xác suất các hành động dẫn đến return thấp:

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi(a_t | s_t; \theta).$$

- **REINFORCE with baseline:** Trừ đi một baseline, thường là $V(s_t)$, để giảm phương sai mà không làm lệch kỳ vọng gradient:

$$\theta \leftarrow \theta + \alpha (G_t - V(s_t)) \nabla_{\theta} \log \pi(a_t | s_t; \theta).$$

- **Actor-Critic:** Kết hợp actor học chính sách $\pi(a|s; \theta)$ và critic học hàm giá trị $V(s; \phi)$ hoặc $Q(s, a; \phi)$. Critic cung cấp tín hiệu lợi thế $A(s, a)$ ổn định hơn cho actor, tạo nền tảng cho các thuật toán hiện đại như PPO.

4.3.2 Thuật toán PPO (Proximal Policy Optimization)

Thuật toán PPO, được giới thiệu bởi OpenAI vào năm 2017, hiện đang là một trong những thuật toán Policy Gradient phổ biến và hiệu quả nhất nhờ khắc phục được nhược điểm của các phương pháp Policy Gradient truyền thống (thường có bước cập nhật quá lớn khiến chính sách bị phá hủy hoàn toàn và không thể phục hồi). PPO giải quyết vấn đề này bằng một cơ chế tinh tế gọi là **Clipping** nhằm giới hạn biên độ thay đổi giữa chính sách cũ (π_{old}) và chính sách mới (π_{θ}) vừa được cập nhật, đảm bảo tác nhân duy trì quá trình học mượt mà, ổn định và không bị "quên" các tri thức quan trọng đã tích lũy.

Hàm mục tiêu Clipped Surrogate Objective đặc trưng của PPO được định nghĩa như sau:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} [\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)]$$

Trong đó, $r_t(\theta) = \frac{\pi(a_t | s_t; \theta)}{\pi(a_t | s_t; \theta_{\text{old}})}$ đại diện cho tỷ lệ xác suất giữa chính sách mới và chính sách cũ. Nếu tỷ lệ này dịch chuyển quá xa khỏi khoảng an toàn $[1 - \epsilon, 1 + \epsilon]$ (với ϵ thường được thiết lập quanh mức 0.1 hoặc 0.2), hàm clip sẽ lập tức kích hoạt để cắt bớt phần thay đổi quá đà, triệt tiêu động lực cập nhật cực đoan.

Chu trình thuật toán và kiến trúc mã giả của PPO:

1. **Khởi tạo cấu trúc mô hình Actor-Critic:** Tạo lập mạng chính sách $\pi(a|s; \theta)$ (mạng Actor đảm nhận vai trò ra quyết định) và mạng hàm giá trị $V(s; \phi)$ (mạng Critic đảm nhận vai trò đánh giá trạng thái) với các tham số khởi tạo ngẫu nhiên θ và ϕ .
2. **Vòng lặp huấn luyện theo Epoch:** Tại mỗi vòng lặp lớn:
 - **Giai đoạn thu thập dữ liệu (Collect Rollouts):** Chạy chính sách tác nhân hiện tại tương tác trực tiếp với môi trường. Để tối ưu hóa hiệu năng tính toán và gia tăng tính đa dạng dữ liệu, PPO thường được cấu hình huấn luyện đồng thời trên nhiều môi trường song song độc lập (`num_envs parallel environments`). Tác nhân sẽ chọn hành động bằng cách lấy mẫu ngẫu nhiên dựa trên phân phối xác suất sinh ra từ mô hình $\pi(a|s; \theta)$ và đóng gói toàn bộ quỹ đạo dữ liệu thu được bao gồm các bộ dữ liệu chuyển đổi trạng thái cùng log xác suất cũ: $(s, a, r, s', \pi_{\text{old}})$.

- **Giai đoạn tính toán lợi thế (Compute Advantages):** Với mỗi bước thời gian t trong chuỗi dữ liệu thu thập, ta ước lượng giá trị lợi thế $\hat{A}_t$ thông qua cơ chế GAE (Generalized Advantage Estimation):

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

Trong đó, $\delta_t = r_t + \gamma V(s_{t+1}; \phi) - V(s_t; \phi)$ là sai số TD Error đo đạc bởi mạng Critic. Tham số điều hòa $\lambda \in [0, 1]$ đóng vai trò kiểm soát, cân bằng linh hoạt giữa độ lệch (bias) và phương sai (variance) của mô hình ước lượng.

- **Giai đoạn cập nhật chính sách (Actor Update):** Thực hiện vòng lặp K lần trên các minibatch trích xuất ngẫu nhiên từ quỹ đạo dữ liệu để tối ưu hóa tham số mạng Actor:

(a) Tính toán giá trị hàm mất mát PPO Loss: $L^{\text{PPO}} = -\mathbb{E}_t \left[\min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$

(b) Cập nhật bộ trọng số θ thông qua phương pháp thúc đẩy gradient descent.

- **Giai đoạn cập nhật hàm giá trị (Critic Update):** Thực hiện vòng lặp K' lần song song để tối ưu hóa tham số mạng Critic:

(a) Tính toán sai số bình phương trung bình (MSE Loss) giữa giá trị mạng Critic dự đoán với giá trị mục tiêu thực tế thu hồi từ môi trường: $L^V = \mathbb{E}_t [(V(s_t; \phi) - (r_t + \gamma V(s_{t+1}; \phi)))^2]$.

(b) Cập nhật bộ trọng số ϕ bằng thuật toán gradient descent nhằm thu hẹp khoảng cách đánh giá.

Mã giả PPO dạng cấu trúc thuật toán Python:

```
1 Initialize policy pi(a|s; theta) and value function V(s; phi)
2 for each epoch:
3     # Bước 1: Thu thập rollouts từ các môi trường song song
4     trajectories = []
5     for env in parallel_environments:
6         s = env.reset()
7         while not terminal(s):
8             # Lấy mẫu hành động dựa trên phân phối xác suất
9             a ~ pi(a|s; theta)
10            r, s_prime = env.step(a)
11            trajectories.append((s, a, r, s_prime, log_prob))
12            s = s_prime
13
14    # Bước 2: Ước lượng hàm lợi thế Advantage bằng GAE
15    for each (s, a, r, s_prime) in trajectories:
16        A = 0
17        for t in trajectory_length downto 0:
18            delta = r[t] + gamma * V(s[t+1]; phi) - V(s[t]; phi)
19            A = delta + gamma * lambda * A
20            advantage[t] = A
```

```
21
22 # Buoc 3: Cap nhat mang chinh sach Actor K lan
23 for k in range(K):
24     for minibatch in trajectories:
25         r_t = pi(a|s; theta) / pi_old(a|s)
26         surrogate1 = r_t * advantage
27         surrogate2 = clip(r_t, 1-eps, 1+eps) * advantage
28         loss_policy = -min(surrogate1, surrogate2)
29         Update theta using gradient descent on loss_policy
30
31 # Buoc 4: Cap nhat mang danh gia Critic K_prime lan
32 for k_prime in range(K_prime):
33     for minibatch in trajectories:
34         target = r + gamma * V(s_prime; phi)
35         loss_value = (V(s; phi) - target)**2
36         Update phi using gradient descent on loss_value
```

Code 2. PPO pseudocode

4.3.3 Đánh giá và so sánh thực tiễn

Ưu điểm vượt trội: Thuật toán PPO sở hữu độ ổn định huấn luyện rất cao nhờ có biên giới giới hạn clipping che chắn, giảm thiểu rủi ro đổ vỡ chính sách đột ngột. Thuật toán này rất dễ cấu hình hệ thống siêu tham số và có khả năng tương thích xuất sắc với cả không gian hành động rời rạc lẫn không gian hành động liên tục phức tạp.

Nhược điểm cốt lõi: Điểm hạn chế lớn nhất của PPO nằm ở tính kém hiệu quả về mặt sử dụng mẫu (sample inefficient) khi đặt lên bàn cân so sánh với các thuật toán off-policy như DQN. Do bản chất là một phương pháp học on-policy (hoặc gần on-policy), PPO yêu cầu hệ thống phải liên tục thu thập một lượng khổng lồ các mẫu rollout mới hoàn toàn tại mỗi epoch để phục vụ huấn luyện, đồng thời bắt buộc phải lưu giữ toàn bộ thông tin các rollout cũ tương ứng chỉ để phục vụ cho tác vụ tính toán hàm lợi thế.

4.4 Self-Play

4.4.1 Nguồn gốc và ý tưởng cơ bản

Self-play được sử dụng rộng rãi trong các trò chơi như cờ vua (AlphaGo) và các trò chơi hành động. Ý tưởng là: thay vì huấn luyện agent chỉ chống lại các chiến lược cố định hoặc ngẫu nhiên, thì ta cho nó chơi với các phiên bản cải thiện của chính nó. Khi agent mạnh hơn, đối thủ cũng phải mạnh hơn để tạo ra áp lực cạnh tranh.

Điều này tạo ra một vòng lặp tích cực: agent v_n được huấn luyện bằng cách chơi với agent v_{n-1} (phiên bản trước). Khi v_n đạt một chất lượng nhất định, nó trở thành đối thủ cho agent tiếp theo v_{n+1} . Quá trình này dẫn đến một "evolutionary arms race" nơi các agent liên tục phát triển để đánh bại đối thủ.

4.4.2 Các bước chi tiết và mã giả

1. **Khởi tạo:** Tạo agent chính π_{main} với tham số ngẫu nhiên. Khởi tạo một bộ các agent cũ $\pi_{\text{pool}} = \{\pi_1, \pi_2, \dots\}$ để đóng vai trò đối thủ.
2. **Lặp cho mỗi epoch:**
 - **Chọn đối thủ:** Chọn ngẫu nhiên một agent từ π_{pool} làm đối thủ. Có thể là bản thân agent chính (π_{main}) hoặc một phiên bản cũ.
 - **Chơi trò chơi:** Chạy một trận đấu (hoặc nhiều trận) giữa π_{main} (vai trò agent 1) và agent đối thủ (vai trò agent 2, 3, hoặc 4 trong Bomberman). Thu thập dữ liệu: (s^i, a^i, r^i, s'^i) cho agent 1 và (s^j, a^j, r^j, s'^j) cho agent khác.
 - **Cập nhật agent chính:** Sử dụng dữ liệu từ chơi (ví dụ với PPO hoặc DQN) để huấn luyện π_{main} . Phần thưởng có thể:
 - $r = +1$ nếu agent sống sót và đối thủ chết
 - $r = -1$ nếu agent chết
 - $r = 0.1$ nếu agent sống sót mỗi bước (survive bonus)
 - $r = +0.5$ nếu agent thu thập vật phẩm
 - **Cập nhật pool:** Sau mỗi K epoch (ví dụ $K = 10$), lưu phiên bản hiện tại của π_{main} vào π_{pool} . Nếu pool quá lớn, có thể loại bỏ các phiên bản yếu nhất.

Mã giả Self-Play:

```
1 Initialize pi_main with random parameters
2 pi_pool = [pi_main] # Opponent pool
3 win_rates = {}
4
5 for epoch in range(num_epochs):
6     # Select opponent
7     opponent_id = random.choice(range(len(pi_pool)))
8     pi_opponent = pi_pool[opponent_id]
9
10    # Play games
11    for game in range(num_games_per_epoch):
12        # Randomize agent order to reduce positional bias
13        if random.random() < 0.5:
14            agent1, agent2 = pi_main, pi_opponent
15            collect_from_main = True
16        else:
17            agent1, agent2 = pi_opponent, pi_main
18            collect_from_main = False
19
20        trajectories = play_game(agent1, agent2)
21
22        # Extract main agent data
23        if collect_from_main:
```



```
24         main_data = trajectories[0]
25     else:
26         main_data = trajectories[1]
27
28     # Reward: +1 if won, -1 if lost, +0.1 per step alive
29     game_result = determine_winner(agent1, agent2)
30     if collect_from_main:
31         if game_result == 1: # agent1 (main) won
32             reward = 1.0
33         else:
34             reward = -1.0
35     else:
36         if game_result == 2: # agent2 (main) won
37             reward = 1.0
38         else:
39             reward = -1.0
40
41     # Accumulate training data
42     training_data.extend(main_data)
43
44     # Update pi_main using collected data
45     train_agent(pi_main, training_data)
46
47     # Periodically add current policy to opponent pool
48     if epoch % update_pool_interval == 0:
49         pi_pool.append(copy(pi_main))
50         if len(pi_pool) > max_pool_size:
51             # Remove weakest from pool (optional)
52             pi_pool = pi_pool[-max_pool_size:]
53
54     # Periodically test against fixed opponents
55     if epoch % eval_interval == 0:
56         for test_opponent in pi_pool:
57             win_rate = evaluate(pi_main, test_opponent,
58                               num_eval_games)
59             print(f"Win rate vs opponent: {win_rate}")
```

Code 3. Self-Play training pseudocode

Một điểm quan trọng: trong Bomberman, có thể có 4 agent cùng lúc. Self-play cơ bản chỉ xem xét 2 agent. Để mở rộng, có thể:

- Thêm hai agent fixed (chơi ngẫu nhiên hoặc rule-based) làm “environment obstacles”.
- Hoặc huấn luyện bốn agent song song, mỗi agent phải học cách chơi chống lại ba đối thủ thích nghi.

Self-play giúp agent phát triển các chiến lược tinh vi để đối phó với các đối thủ thích

nghe. Nó tự động gen dữ liệu huấn luyện liên quan. Thách thức là vòng lặp có thể trở nên không ổn định: agent có thể rơi vào các "local optima" (ví dụ chỉ biết một chiến lược lạ nhưng hiệu quả chống lại một đối thủ cụ thể), hoặc một phiên bản yếu trong pool vô tình được chọn quá thường xuyên.

4.5 Behaviour Cloning

4.5.1 Nguồn gốc và ý tưởng cơ bản

Behavior Cloning được khái niệm từ học có giám sát cơ bản. Ý tưởng là: cho một tập hợp demo từ chuyên gia (expert, có thể là con người hoặc các agent tốt có sẵn), huấn luyện một mô hình để dự đoán hành động được chọn bởi chuyên gia ở mỗi trạng thái. Nó là cách nhanh nhất để tạo một agent tương đối hợp lý mà không cần định nghĩa hàm phần thưởng phức tạp hoặc chờ RL hội tụ.

Cách tiếp cận này được sử dụng trong robotics, driving simulators, và các tác vụ học từ demo (learning from demonstrations). Tuy nhiên, nó có một vấn đề cố hữu: nếu agent hoạt động khác so với chuyên gia tại bất kỳ thời điểm nào, nó có thể rơi vào các trạng thái chuyên gia chưa bao giờ gặp phải (out-of-distribution states), và nó phải tự đưa ra hành động tiếp theo.

4.5.2 Các bước chi tiết và mã giả

1. **Thu thập demo:** Ghi lại các trận đấu từ những con người chơi Bomberman khéo léo. Với mỗi frame, lưu trữ:
 - **Trạng thái s :** biểu diễn bản đồ (ví dụ: pixel, feature vector)
 - **Hành động a :** số nguyên từ 0-5 tương ứng với STOP, LEFT, RIGHT, UP, DOWN, PLACE_BOMBSố lượng frames tốt: ít nhất vài chục nghìn frames (tương đương hàng trăm trò chơi).
2. **Xây dựng dataset:** Tổ chức dữ liệu thành cặp (s_i, a_i) và chia thành tập training (ví dụ 80%) và tập validation (20%).
3. **Xây dựng mô hình:** Tạo một mạng nơ-ron $\pi_\theta(a|s)$ nhận trạng thái làm đầu vào và xuất ra phân bố xác suất trên các hành động. Kiến trúc thường dùng:
 - Nếu trạng thái là ảnh: CNN (Convolutional Neural Network)
 - Nếu trạng thái là feature vector: MLP (fully connected layers)
4. **Huấn luyện:** Lặp qua dữ liệu training theo batch. Với mỗi batch:
 - Dự đoán: $\hat{a}_i = \pi_\theta(s_i)$ (chọn hành động có xác suất cao nhất)
 - Tính loss (cross-entropy):

$$\mathcal{L} = - \sum_i \log \pi_\theta(a_i | s_i)$$

- Cập nhật θ bằng gradient descent (ví dụ: Adam optimizer)

Huấn luyện cho đến khi accuracy trên tập validation không cải thiện (early stopping).

5. **Đánh giá:** Kiểm tra độ chính xác trên tập test. Ngoài ra, cho agent chơi vài trận đấu để xem hành vi có hợp lý không.

Mã giả Behavior Cloning:

```
1 # Data collection phase
2 demonstrations = []
3 for each human_game in recorded_games:
4     for each frame in human_game:
5         state = extract_state(frame)
6         action = extract_action(frame) # What human did
7         demonstrations.append((state, action))
8
9 # Split data
10 train_data, val_data = split(demonstrations, test_size=0.2)
11
12 # Build model
13 model = NeuralNetwork(input_dim=state_dim,
14                        output_dim=num_actions)
15
16 optimizer = Adam(learning_rate=0.001)
17
18 # Training
19 best_val_loss = infinity
20 patience_counter = 0
21 max_patience = 10
22
23 for epoch in range(max_epochs):
24     total_train_loss = 0
25     for batch in train_data:
26         states, actions = batch
27
28         # Forward pass
29         logits = model.forward(states) # Shape: [batch_size,
30                                         num_actions]
31
32         # Cross-entropy loss
33         loss = cross_entropy_loss(logits, actions)
34
35         # Backward pass
36         grads = compute_gradients(loss)
37
38         # Update
39         model.update(grads, optimizer)
40
41     total_train_loss += loss
```



```
40     avg_train_loss = total_train_loss / len(train_data)
41
42     # Validation
43     val_loss = 0
44     val_correct = 0
45     for batch in val_data:
46         states, actions = batch
47         logits = model.forward(states)
48         loss = cross_entropy_loss(logits, actions)
49         val_loss += loss
50
51         predicted_actions = argmax(logits, axis=1)
52         val_correct += sum(predicted_actions == actions)
53
54     avg_val_loss = val_loss / len(val_data)
55     val_accuracy = val_correct / total_val_samples
56
57     print(f"Epoch {epoch}: train_loss={avg_train_loss:.4f}, "
58           f"val_loss={avg_val_loss:.4f}, "
59           f"val_acc={val_accuracy:.4f}")
60
61     # Early stopping
62     if avg_val_loss < best_val_loss:
63         best_val_loss = avg_val_loss
64         patience_counter = 0
65         save_model(model) # Save best model
66     else:
67         patience_counter += 1
68         if patience_counter >= max_patience:
69             print("Early stopping")
70             break
71
72     # Load best model
73     model = load_best_model()
74
75     # Deployment: agent plays games
76     for game in range(num_test_games):
77         s = game.reset()
78         while not terminal(s):
79             logits = model.forward(s)
80             a = argmax(logits) # Greedy action selection
81             s = game.step(a)
```

Code 4. Behavior Cloning pseudocode

Ưu điểm của BC: nhanh chóng, dễ triển khai, không cần định nghĩa hàm phần thưởng.
Nhược điểm: phụ thuộc vào chất lượng demo, agent không thể vượt quá kỹ năng chuyên gia, dễ gặp distribution shift.

Nhược điểm: BC dễ gặp phải vấn đề là nếu agent đưa ra quyết định khác so với chuyên gia ở bước t , nó rơi vào trạng thái s'_t mà chuyên gia chưa bao giờ gặp trong dữ liệu demo (Distribution shift). Khi agent dự đoán hành động ở s'_t , nó dựa vào một trạng thái out-of-distribution, lỗi có thể tích lũy theo thời gian (error compounding).

Một cải tiến gọi là DAgger (Dataset Aggregation) có thể giải quyết vấn đề này:

- Huấn luyện BC trên dữ liệu demo ban đầu.
- Chạy agent và ghi lại các trạng thái nó gặp phải.
- Yêu cầu chuyên gia ghi lại hành động tại các trạng thái mới này.
- Thêm dữ liệu mới vào dataset và huấn luyện lại.
- Lặp lại.

4.5.3 Monte Carlo Tree Search (MCTS)

Trong số các phương pháp hoạch định trực tuyến (online planning) thuộc nhánh học tăng cường dựa trên mô hình (Model-based RL), Tìm kiếm cây Monte Carlo (Monte Carlo Tree Search - MCTS) đóng vai trò là một thuật toán xấp xỉ đồ thị quyết định cực kỳ mạnh mẽ. Khác với các thuật toán tìm kiếm duyệt cạn truyền thống vốn bị giới hạn bởi bùng nổ tổ hợp do không gian trạng thái quá lớn, MCTS kết hợp linh hoạt khả năng định hướng của cây quyết định với tính chất ngẫu nhiên mang tính thống kê của các mô phỏng Monte Carlo. Thuật toán tiến hành xây dựng một cây tìm kiếm bất đối xứng một cách tăng trưởng thông qua chu kỳ lặp đi lặp lại của bốn bước cốt lõi: Lựa chọn (Selection), Mở rộng (Expansion), Mô phỏng (Simulation/Playout), và Lan truyền ngược (Backpropagation).

Quy trình thuật toán chi tiết và nền tảng toán học:

1. **Selection (Lựa chọn):** Tiến trình bắt đầu từ nút gốc v_0 (đại diện cho trạng thái hiện tại của môi trường). Thuật toán duyệt qua các nút con nội bộ của cây bằng cách áp dụng một chính sách chọn lọc cây (tree policy). Để giải quyết bài toán cốt lõi về việc cân bằng giữa khai thác tri thức hiện tại (exploitation) và khám phá các vùng không gian mới (exploration), thuật toán UCT (Upper Confidence Bounds for Trees) - một nhánh mở rộng của bài toán Multi-armed Bandit áp dụng tiêu chuẩn UCB1 - thường được sử dụng phổ biến nhất:

$$UCT(v) = \frac{Q(v)}{N(v)} + C \sqrt{\frac{\ln N(\text{parent})}{N(v)}}$$

Trong đó:

- $Q(v)$ là tổng giá trị phần thưởng tích lũy thu được tại nút v qua các lượt mô phỏng trước đó.
- $N(v)$ là tổng số lần nút v đã được ghé thăm trong toàn bộ tiến trình tìm kiếm.
- $N(\text{parent})$ biểu diễn số lần nút cha trực tiếp của nút v được duyệt qua.

- C là hằng số điều hòa khám phá (thường được thiết lập về mặt lý thuyết là $C = \sqrt{2}$ khi giá trị phần thưởng được chuẩn hóa về khoảng $[0, 1]$).

Ý nghĩa toán học của biểu thức UCT chia làm hai thành phần tường minh: số hạng đầu tiên $\frac{Q(v)}{N(v)}$ đại diện cho giá trị kỳ vọng trung bình (giá trị khai thác), ưu tiên các nhánh đi có hiệu suất cao; số hạng thứ hai chứa căn thức đại diện cho độ bất định toán học (giá trị khám phá), tỷ lệ nghịch với số lần nút đó được thăm. Thuật toán liên tục chọn nút con có chỉ số UCT lớn nhất cho đến khi chạm vào một nút lá v_l (nút chưa được mở rộng hoàn toàn hoặc chứa các hành động chưa từng được khám phá).

2. **Expansion (Mở rộng):** Khi tiến trình lựa chọn dừng lại tại nút lá v_l , nếu trạng thái này chưa phải là trạng thái kết thúc trận đấu (terminal state), thuật toán sẽ tiến hành mở rộng cây tìm kiếm. Một hoặc nhiều nút con mới v_c sẽ được khởi tạo và liên kết vào cây, tương ứng với các hành động hợp lệ khả thi mà tác nhân có thể thực hiện từ trạng thái của v_l . Để tối ưu hóa tài nguyên tính toán trong các môi trường có hệ số rẽ nhánh (branching factor) lớn, hệ thống thông thường chỉ thêm một nút con duy nhất cho mỗi chu kỳ.
3. **Simulation (Mô phỏng / Playout):** Từ nút con mới được thiết lập v_c , một phiên chơi thử nghiệm giả lập (rollout) sẽ được kích hoạt và chạy liên tục cho đến khi trò chơi đạt điều kiện kết thúc hoặc chạm tới một độ sâu giới hạn quy định. Trong giai đoạn này, một chính sách mặc định (default policy) - có thể là chọn hành động ngẫu nhiên hoàn toàn (uniform random) hoặc bán ngẫu nhiên kết hợp các luật heuristic đơn giản - sẽ điều khiển các thực thể di chuyển. Mục tiêu của bước này là thu về một ước lượng nhanh, dù có thể chứa nhiều nhiễu thống kê, về giá trị tiềm năng lâu dài của nhánh quyết định vừa chọn mà không cần cấu hình hàm đánh giá trạng thái phức tạp.
4. **Backpropagation (Lan truyền ngược):** Ngay khi phiên mô phỏng kết thúc và trả về tín hiệu kết quả (reward), dữ liệu thống kê này sẽ được truyền ngược theo lộ trình tuyến tính từ nút mô phỏng v_c lên tới tận nút gốc v_0 . Với mỗi nút v nằm trên trục đường truyền ngược này, hệ thống tiến hành cập nhật đồng thời các đại lượng:

$$N(v) \leftarrow N(v) + 1$$

$$Q(v) \leftarrow Q(v) + \text{reward}$$

Tùy thuộc vào bản chất môi trường (đơn tác nhân, đa tác nhân hợp tác hay đối kháng), hàm phần thưởng (reward) có thể được cấu hình linh hoạt. Trong kịch bản đối kháng zero-sum hai người chơi, giá trị cập nhật thường đảo dấu tuần tự giữa các tầng để phản ánh tư duy Minimax, hoặc sử dụng hàm điểm số phức tạp liên tục dựa trên các tiêu chí cụ thể.

5. **Chu kỳ lặp và Ra quyết định cuối cùng:** Toàn bộ chuỗi bốn bước từ (1) đến (4) được vận hành liên tục dưới dạng một vòng lặp anytime algorithm. Tiến trình tìm kiếm có thể dừng lại ở bất kỳ thời điểm nào tùy thuộc vào giới hạn tài nguyên (như giới hạn thời gian tính toán real-time hoặc số lượt lặp cố định). Khi vòng lặp kết thúc, tác nhân sẽ chính thức đưa ra quyết định hành động thực tế bằng cách chọn

nút con trực tiếp của nút gốc sở hữu số lần ghé thăm cao nhất ($\max N(v)$ - Robust Child) hoặc sở hữu giá trị hiệu suất trung bình lớn nhất ($\max \frac{Q(v)}{N(v)}$ - Max Child).

Tính hội tụ toán học của MCTS: Một trong những bảo chứng lý thuyết quan trọng nhất của MCTS (đặc biệt là biến thể UCT) là *tính hội tụ tiệm cận*. Khi số lượt mô phỏng tiến dần đến vô hạn ($N \rightarrow \infty$), xác suất tác nhân lựa chọn các nhánh đi suboptimal (dưới tối ưu) sẽ hội tụ về 0 nhờ có thành phần khám phá của UCT che chắn. Cây tìm kiếm MCTS sẽ tăng trưởng bất đối xứng, tập trung sâu vào các vùng không gian chiến lược tối ưu, và giá trị ước lượng $\frac{Q(v)}{N(v)}$ sẽ hội tụ chính xác về giá trị trò chơi lý thuyết Minimax (đối với game đối kháng) hoặc phương trình tối ưu Bellman.

Ứng dụng trong môi trường chiến thuật Bomberman:

Khi triển khai MCTS cho trò chơi Bomberman, đồ thị cây sẽ mô phỏng các chuỗi hành động kết hợp tuần tự trong tương lai của tác nhân bao gồm: Di chuyển (Lên, Xuống, Trái, Phải), Đứng yên (Chờ đợi), hoặc Đặt bom. Ưu điểm cốt lõi của MCTS trong ngữ cảnh này là tính độc lập dữ liệu huấn luyện (training data-free) và khả năng thích ứng linh hoạt với các thay đổi động của bản đồ. Thuật toán không đòi hỏi một mô hình học sâu được huấn luyện trước tốn kém mà chỉ cần tích hợp một bộ giả lập công cụ trò chơi (forward model/game engine) có khả năng tính toán trạng thái kế tiếp cực nhanh. Tại mỗi bước đi thực tế, cây tìm kiếm được tái cấu trúc hoàn toàn từ trạng thái hiện tại, giúp tác nhân phản ứng nhạy bén trước sự dịch chuyển bất ngờ của kẻ địch hay các chuỗi nổ dây chuyền của bom.

Tuy nhiên, cấu trúc thuần túy của MCTS vấp phải những rào cản kỹ thuật nghiêm trọng khi đối mặt với tính chất thời gian thực và độ trễ nổ của bom:

- **Điểm nghẽn tính toán mô phỏng (Simulation Bottleneck):** Để đưa ra quyết định có độ chính xác cao trong môi trường có hệ số rẽ nhánh lớn và thời gian nổ kéo dài (bom hẹn giờ cần nhiều bước để kích nổ), MCTS cần phải thực hiện hàng ngàn đến hàng triệu lượt mô phỏng ngẫu nhiên sâu. Điều này tạo áp lực cực lớn lên CPU và khó lòng đáp ứng tần suất ra quyết định real-time (thường dưới 100ms mỗi khung hình).
- **Tật mù quán tính của Playout ngẫu nhiên (Tactical Blindness):** Do bước mô phỏng sử dụng chính sách ngẫu nhiên, các tác nhân giả lập trong Playout thường di chuyển không mục đích. Điều này dẫn đến việc ước lượng sai lệch: một vị trí đặt bom chiến thuật có thể bị đánh giá thấp chỉ vì chính sách ngẫu nhiên trong Playout điều khiển tác nhân tự sát; hoặc ngược lại, một nhánh đi nguy hiểm bị đánh giá cao do sự may mắn ngẫu nhiên của phiên Playout không kích hoạt bẫy nổ.

Xu hướng tinh chỉnh hiện đại - PUCT và Sự kết hợp Mạng nơ-ron:

Để khắc phục các nhược điểm trên, các hệ thống AI hiện đại (được truyền cảm hứng từ kiến trúc AlphaGo/AlphaZero) thường loại bỏ bước Playout ngẫu nhiên truyền thống và thay thế bằng việc kết hợp với các mạng nơ-ron tham số. Khi đó, biểu thức lựa chọn UCT được chuyển đổi thành thuật toán **PUCT (Predictor + UCB for Trees)**:



$$\text{PUCT}(v) = \frac{Q(v)}{N(v)} + C \cdot P(s, a) \cdot \frac{\sqrt{N(\text{parent})}}{1 + N(v)}$$

Trong đó, $P(s, a)$ đóng vai trò là phân phối xác suất tiên nghiệm (prior probability) được cung cấp từ một mạng chính sách (Policy Network) đã qua huấn luyện. Mạng chính sách này giúp thu hẹp lập tức hệ số rẽ nhánh bằng cách chỉ hướng xung lực tìm kiếm vào các hành động có triển vọng cao. Song song đó, giá trị phần thưởng cuối cùng tại nút lá thay vì chạy Playout ngẫu nhiên kéo dài sẽ được dự đoán trực tiếp thông qua một mạng giá trị (Value Network) $V(s)$. Sự kết hợp hoàn hảo này biến MCTS từ một thuật toán tìm kiếm mù quáng thành một tiến trình hoạch định có định hướng cao, giải phóng đáng kể năng lực tính toán và nâng tầm tư duy chiến thuật của tác nhân lên mức độ chuyên nghiệp.

5. Nguồn tham khảo

1. [Khóa CS234 của Stanford về Reinforcement Learning](#)
2. [RL Book, Sutton Barto](#)
3. [RL Course Toronto University](#)
4. [Neuriton](#)
5. [RL Exercises, AI VIETNAM](#)
6. [Single-File RL Algorithms Code](#)